

Documentation StartkitV1

Documentation — StartkitV1

Documentation du projet **StartkitV1** : structure, architecture, utilisation et script de génération d'applications.

1. Présentation

StartkitV1 est un **squelette Angular** réutilisable qui fournit :

- Une **architecture** en trois couches : core, shared, features.
- Des **layouts** prêts à l'emploi : main-guest (auth), main-layout (back office), main-portal (portail, utilisé uniquement par le script de génération).
- Une **page de connexion** et des services d'auth (sans protection des routes par défaut : toutes les pages sont accessibles sans connexion).
- Un **script** pour créer une nouvelle application à partir du Startkit en choisissant portail et/ou back-office.

Le Startkit est pensé pour ne pas bloquer l'accès au front : **aucun guard n'est appliqué** sur les routes ; vous pouvez activer l'auth plus tard en réintroduisant les guards.

2. Architecture

2.1 Règles de découpage

Dossier	Rôle
core/	Services globaux (auth, API, notification), guards, interceptors HTTP, constantes. Pas de logique métier spécifique.
shared/	Composants réutilisables et sans métier : layouts (main-guest, main-layout, main-portal), pipes (ex. date-fr).
features/	Modules métier, un par domaine : auth (login), admin (back office), accueil (portail, présent pour le script create-app). Chaque feature a un routes.ts et des composants.

2.2 Layouts

- **main-guest** : enveloppe les pages d'authentification (ex. /auth — formulaire de connexion).
- **main-layout** : enveloppe le back office (sidebar + zone de contenu), utilisé pour /admin.
- **main-portal** : enveloppe le portail (front office) ; présent dans le code mais **non utilisé** dans les routes par défaut du Startkit. Utilisé lorsque l'on génère une app avec l'option portail via le script.

2.3 Arborescence type

```
src/app/
├── core/
│   ├── constants/ # Messages d'erreur, etc.
│   ├── guards/ # auth.guard, guest.guard (non utilisés par défaut)
│   ├── interceptors/ # auth (token), error-notification
│   └── services/ # auth, api, notification
├── shared/
│   ├── components/ # main-guest, main-layout, main-portal
│   └── pipes/ # date-fr
├── features/
│   ├── auth/ # Login (main-guest)
│   ├── admin/ # Back office (main-layout)
│   └── accueil/ # Portail (main-portal), pour create-app
├── app.routes.ts
├── app.config.ts
└── app.component.ts
```

3. Routes (comportement par défaut)

Aucun **guard** n'est appliqué : toutes les URLs sont accessibles sans connexion.

Route	Layout	Contenu
/	—	Redirection vers /admin
/auth	main-guest	Page de connexion (login)
/admin	main-layout	Back office (tableau de bord, etc.)
**	—	Redirection vers /admin

Pour protéger des routes (ex. /admin réservé aux utilisateurs connectés), il suffit d'ajouter canActivate: [AuthGuard] sur les routes concernées et d'importer AuthGuard depuis core/guards/auth.guard.

4. Authentification

- **AuthService** : login, logout, token, isLoggedInIn, currentUser. Stockage du token en localStorage.
- **auth.interceptor** : ajoute le header Authorization: Bearer <token> aux requêtes HTTP (sauf vers /auth/).
- **error-notification.interceptor** : en cas d'erreur 401, affiche un message et appelle auth.logout() (redirection vers /auth).
- **Guards** : authGuard et guestGuard existent dans core/guards/ mais ne sont **pas** utilisés dans les routes par défaut.

La base URL de l'API est définie dans src/environments/environment.development.ts (apiUrl). En production, environment.ts est utilisé (remplacement configuré dans angular.json).

5. Démarrage et build

Prérequis

- Node.js 20 LTS (recommandé)
- npm

Commandes

Installation des dépendances

npm install

Serveur de développement (http://localhost:4200)

npm start

Build de production

npm run build

Les artefacts de build sont dans dist/startkitv1/.

6. Créer une application à partir du Startkit

Le script **create-app** (présent uniquement dans le Startkit) permet de générer une nouvelle application en précisant le **nom** et si l'app aura un **portail** et/ou un **back-office**.

Emplacement

- Script : scripts/create-app.js
- Documentation du script : scripts/README.md

Utilisation

Mode interactif (questions : nom, portail ?, back-office ?)

```
npm run create-app:interactive
```

Ligne de commande

```
npm run create-app -- --name mon-app
```

```
npm run create-app -- --name mon-app --back-office-only
```

```
npm run create-app -- --name mon-app --portail-only
```

```
npm run create-app -- --name mon-app --portail --back-office
```

```
npm run create-app -- --name mon-app --out-dir ../mes-projets
```

L'application est créée dans <out-dir>/<nom> (par défaut au-dessus du dossier Startkit). Le dossier scripts/ et les scripts npm create-app ne sont **pas** copiés dans l'app générée.

7. Environnements

- **Développement** : src/environments/environment.development.ts — apiUrl, production: false.
- **Production** : src/environments/environment.ts — utilisé en build prod via fileReplacements dans angular.json.

Adapter apiUrl selon votre backend.

8. Thème et styles

Les variables CSS globales sont dans src/styles.css (ex. --app-primary, --app-surface, --app-bg, --app-text). Vous pouvez les surcharger pour adapter la charte graphique sans modifier le core.

9. Ajouter une feature

1. Créer un dossier sous src/app/features/<nom-feature>/ (ex. features/mes-pages/).
2. Y ajouter un routes.ts qui exporte des Routes (ex. MES_PAGES_ROUTES).
3. Créer les composants dans features/<nom-feature>/components/.
4. Dans app.routes.ts, ajouter une route avec loadChildren vers ce routes.ts, et l'associer au layout souhaité (main-layout pour le back office, main-portal pour le portail si vous l'utilisez).

Convention : nom de feature en **kebab-case**, export de routes en XXX_ROUTES.

10. Références

- **Guide d'architecture** (workspace parent) : GUIDE-STARTKIT-ANGULAR-PORTAL-BACKOFFICE.md
- **Script create-app** : scripts/README.md